

Fuzzy Visual Obstacle Avoidance Using OpenCV and iRobot Create3

Gabriel Ferrer¹

I. INTRODUCTION

We are in the process of adopting iRobot Create3 robots [2] for our undergraduate robotics course. This work specifically addresses teaching Computer Vision as a sensing technique. We have developed a fuzzy-logic-based reactive controller that uses OpenCV's contour detector to identify the boundary between the ground and objects resting upon it, assuming a flat ground area of uniform appearance [1]. The robot drives towards the highest open space in the image.

II. EQUIPMENT AND SETUP

We are using a Raspberry Pi 4 Model B with 4 GB RAM and 128 GB MicroSD card for persistent storage. It is connected to an iRobot Create3 robot [2] using a USB-C cable. It is equipped with a Logitech C270 webcam.

The robot and the Raspberry Pi communicate using ROS2 Humble Hawksbill. The controller is written in Python. Computer vision code uses OpenCV 4.8.0 [3].

III. IMPLEMENTATION DETAILS

A. Image Processing

We process each image using OpenCV [3] as follows:

- Use `cv2.resize()` to resize the image to 640 x 480.
- Use `cv2.cvtColor()` to convert the image to grayscale to prepare for thresholding.
- The carpet in the testing area is not uniform in color. It is largely gray, but speckled with numerous dark-colored dots. To eliminate these artifacts, we use `cv2.morphologyEx()` with the `cv2.MORPH_CLOSE` option (dilation followed by erosion) with a 9x9 kernel. It has been very effective in eliminating the noise from the carpet speckling.
- We then threshold the image using `cv2.threshold(filtered, 127, 255, 0)`.
- We then invoke the `cv2.findContours()` function with the `CHAIN_APPROX_NONE` parameter, ensuring that it returns a dense collection of contour points.
- As we seek the boundary between floor and not-floor from the robot's perspective, we find the largest y value for each x value among the contour points.
- To subdivide the resulting contour points into distinct objects, we cluster into groups as follows:
 - Sort points in increasing order of x coordinate.
 - Traverse the points. When the y coordinate value changes by more than 1 pixel, save the current group and start a new group.

- The **height** of each cluster is defined to be its largest y coordinate. The best cluster is defined as the cluster with the smallest height (i.e., y coordinate that is physically high in the image) that exceeds 20 pixels in width.
- Once the image processing is complete, we use the best cluster to determine the robot's target steering direction.

B. Fuzzy Logic

Given the best cluster as calculated above, we determine a target steering direction by averaging the x values from that best cluster. As that target direction is expressed in terms of pixels, we use fuzzy logic in a two-step process to convert that target direction into motor instructions.

First, we fuzzify the average x value into a falling fuzzy value, where an x value of 0 fuzzifies at 1.0 and a value of 640 fuzzifies at 0.0. Then, we defuzzify the value into an angular z value in a ROS2 `Twist` message [2] ranging from -0.784 to 0.784 radians. This results in a left turn of approximately 45 degrees when near 1.0. Values near zero produce corresponding right turns.

IV. RESULTS AND FUTURE WORK

So far, the robot drives in very smooth paths, finding an area of maximum space. Unfortunately, when caught in a corner, it struggles to escape. Future work ideas include:

- We will explore integrating the IR and bump sensors to help the robot escape corners.
- The selected area of maximum space often oscillates between different regions of the image. We plan to explore maintaining focus on a previously chosen region, in order to reduce these oscillations.
- We would like to use the ground contours to perform object segmentation. This could be used for object recognition, and could potentially serve as an input for a visual SLAM algorithm.
- We plan to expand the inventory of pertinent Computer Vision techniques for this task, in order to give students multiple options for exercises of this kind.

Source code is available on GitHub [4].

REFERENCES

- [1] Horswill, I. (1993). Polly: a vision-based artificial agent. In *Proceedings of the National Conference on Artificial Intelligence* (pp. 824-829).
- [2] https://iroboteducation.github.io/create3_docs/api/ros2/. iRobot Create3 ROS2 API. Retrieved July 15, 2023.
- [3] <https://pypi.org/project/opencv-python/>. OpenCV for Python. Retrieved July 15, 2023.
- [4] https://github.com/gjf2a/ros2_create3_examples. Source code. Retrieved July 15, 2023.

¹Gabriel Ferrer is Professor of Computer Science, Hendrix College, Conway, AR 72032, USA ferrer@hendrix.edu